

A photograph of the Aurora Borealis (Northern Lights) over a dark lake at night. The aurora displays vibrant green and blue light curtains against a starry sky. The lights are reflected in the calm water below. Silhouettes of mountains and hills are visible in the background.

Porting Microsoft Aurora to Dawn

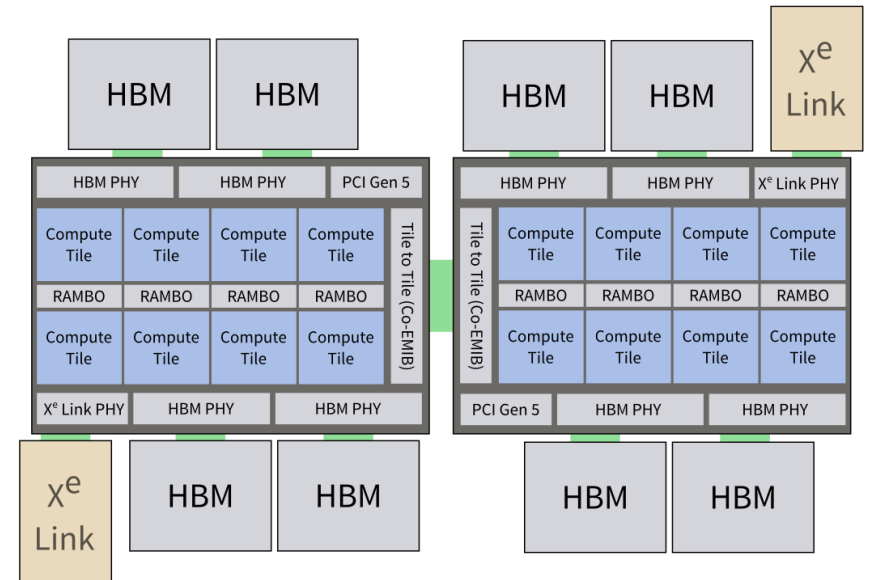
CoSeC 2025

3rd December 2025

Rosie Wood, Iain Stenson, David Llewellyn-Jones
(Research Computing team at The Alan Turing Institute)

Dawn

- AIRR – “AI-optimised HPC”
- Intel® Data Center GPU Max 1550 GPUs
- Each tile offers 64 GB memory (i.e. 128 GB across the full GPU)
- *flat* or *composite* device hierarchy



Microsoft Aurora

- “*A foundation model for the Earth System*”
- Developed at Microsoft Research Cambridge, including Turing researchers.
- It was trained on Azure using A100 GPUs
- Requires GPUs with around 80GB memory and/or distributed training.



What is different when using Intel GPUs?

- Intel oneAPI versus CUDA
 - Intel oneCCL replaces NCCL for communications/networking
 - Intel oneMKL/oneDNN replaces cuBLAS/cuDNN for maths/linear algebra
- Intel extension for PyTorch (` ipex `)
- Move model/data/etc to XPU:
 - ``model.to("xpu")`` vs ``model.to("cuda")``
 - ``data.to("xpu")`` vs ``data.to("cuda")``

What is different when using Intel GPUs?

```
echo "## Loading modules"
```

```
module purge
```

```
module load default-dawn
```

```
module load lua
```

```
module load intel-oneapi-ccl/2021.14.0
```

```
module load intel-oneapi-mpi/2021.14.1
```

```
module load intel-oneapi-mkl/2025.0.1
```

```
# load intel oneapi compilers (gives us sycl-ls command)
```

```
module load intel-oneapi-compilers/2025.0.3/gcc/sb5vj5us
```

```
echo
```

```
echo "## Configuring environment"
```

```
VENV_DIR=../../dawn/environments/venv_3_11_11
```

```
# Merge tiles into full devices, for extra memory.
```

```
export ZE_FLAT_DEVICE_HIERARCHY=COMPOSITE
```

Load Intel oneAPI modules

Set device hierarchy

```
pip install --upgrade pip
```

```
pip install -e ../../[dawn]
```

```
pip install torch==2.7.0 torchvision==0.22.0 torchaudio==2.7.0 --index-url https://download.pytorch.org/whl/xpu
```

```
pip install --trusted-host pytorch-extension.intel.com intel-extension-for-pytorch==2.7.10+xpu onecccl_bind_pt==2.7.0+xpu  
--extra-index-url https://pytorch-extension.intel.com/release-whl/stable/xpu/us/
```

Install pytorch and
ipex

```
python inference.py --d ../era5/era_v_inf/ -n 28 --save -o preds_${i}.pkl > inference_28_steps_${i}.txt &
```

Run inference.py

What is different when using Intel GPUs?

```
import argparse
import logging
import sys
import time
import json

from pathlib import Path

import torch

from aurora_hpc.dataset import AuroraDataset

def main(
    download_path: str,
    nsteps: int,
    start_index: int = 1,
    save: bool = False,
    output_file: str = "preds.pkl",
    **kwargs,
):
    """Run inference on the Aurora model for a specified number of steps.

    Parameters
    download_path : str
        Path to the directory containing the downloaded data.
    nsteps : int
        Number of steps to infer ahead.
    start_index : int, optional
        Index to start the rollout from, by default 1
    save : bool, optional
        Whether to save outputs, by default False
    output_file : str, optional
        Output file name if saving outputs, by default "preds.pkl"
    kwargs : dict
        kwargs to pass to the AuroraDataset constructor (e.g. if you want to add filepaths)
    """
    time_start_total = time.time()

    print("Loading data...")
    dataset = AuroraDataset(
        data_path=download_path,
        **kwargs
    )
    # Defaults to the 2023-01-01 dataset. Use kwargs to specify other files.
    print("Getting input batch...")
    batch = dataset[start_index:0] # Get the first item in the dataset.

    from aurora import Aurora, rollout

    print("Loading model")
    model = Aurora(use_lora=False) # The pretrained version does not use LoRA.
    model.load_checkpoint("microsoft/aurora", "aurora-0.25-pretrainedckpt")

    model.eval()
    model = model.to("cuda")

    print("doing rollout")
    preds = []
    times = []

    with torch.inference_mode():
        time_start = time.time()
        for pred in rollout(model, batch, steps=nsteps):
            preds.append(pred.to("cpu"))
            time_end = time.time()
            print(f"Time for one step: (time_end - time_start)")
            times.append(time_end - time_start)
            time_start = time.time()

    avg_time = sum(times[1:]) / len(times[1:]) # Exclude the first step time
    print(f"Average time for last (nsteps - 1) steps: (avg_time)")
    print(f"Total time for (nsteps) steps: (sum(times))")

    import pickle

    time_end_total = time.time()
    print(f"Total time: (time_end_total - time_start_total)")

    if save:
        print(f"Saving predictions to {output_file}")
        with open(output_file, "wb") as f:
            pickle.dump(preds, f)
```

← Import `ipex`

```
12
13+ print("Importing ipex")
14 from pathlib import Path
15
16+ import intel_extension_for_pytorch
  + as ipex
17 import torch
```

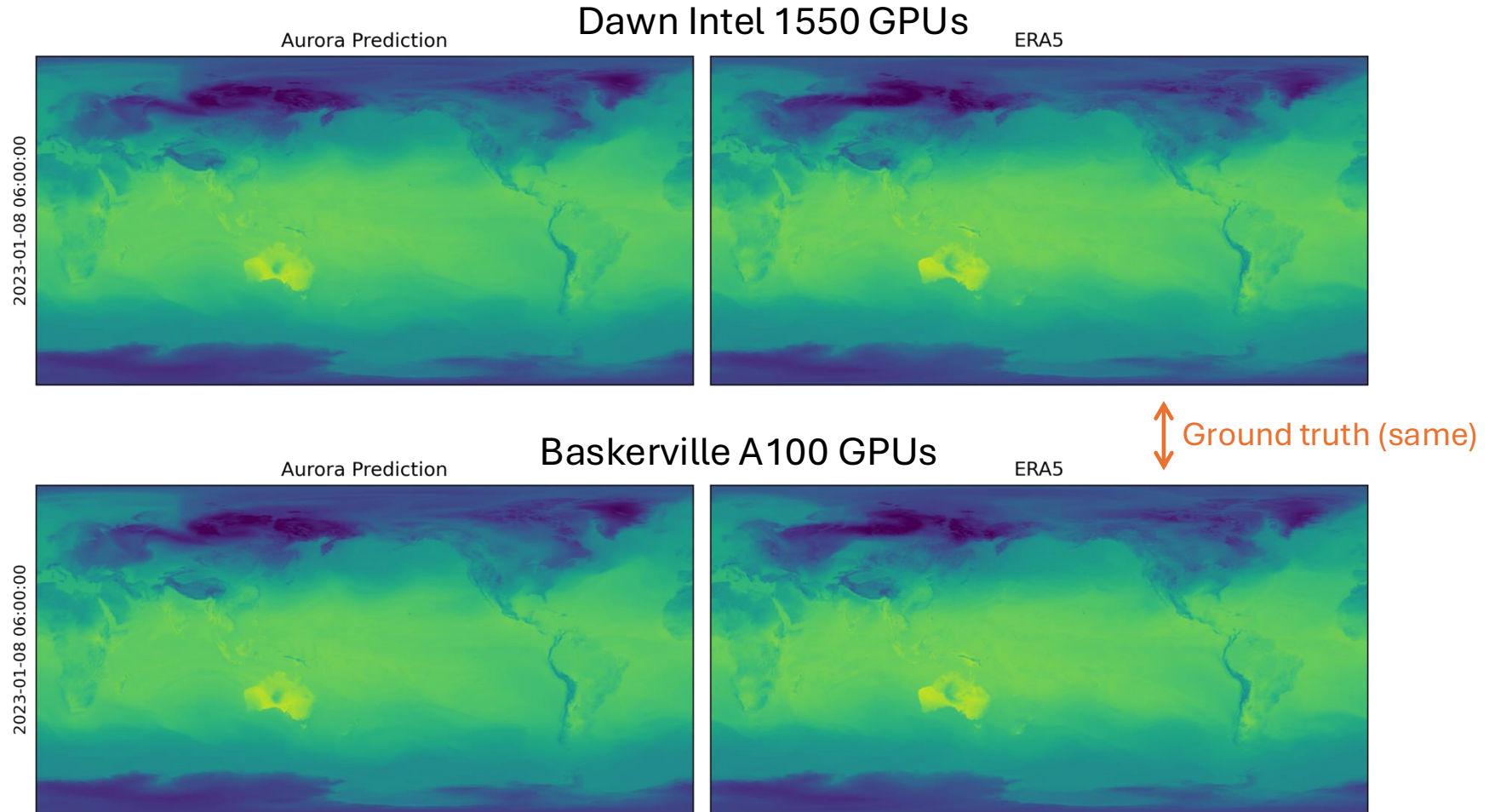
← Move model (data, etc) to `xpu`

```
63
64 model.eval()
65+ model = model.to("xpu")
66
```

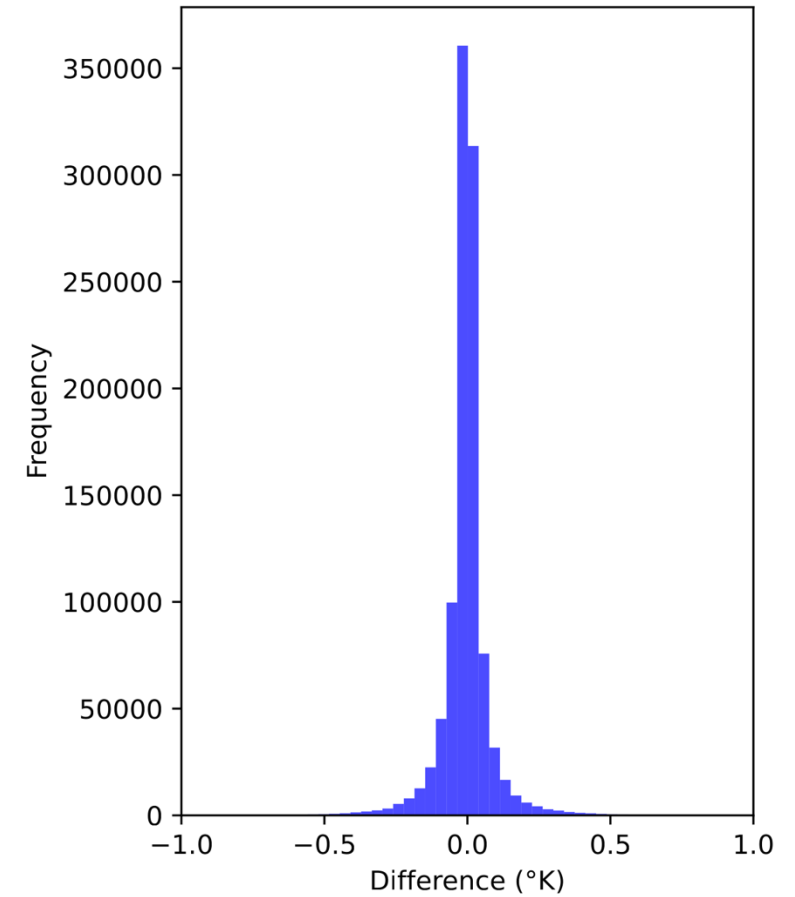
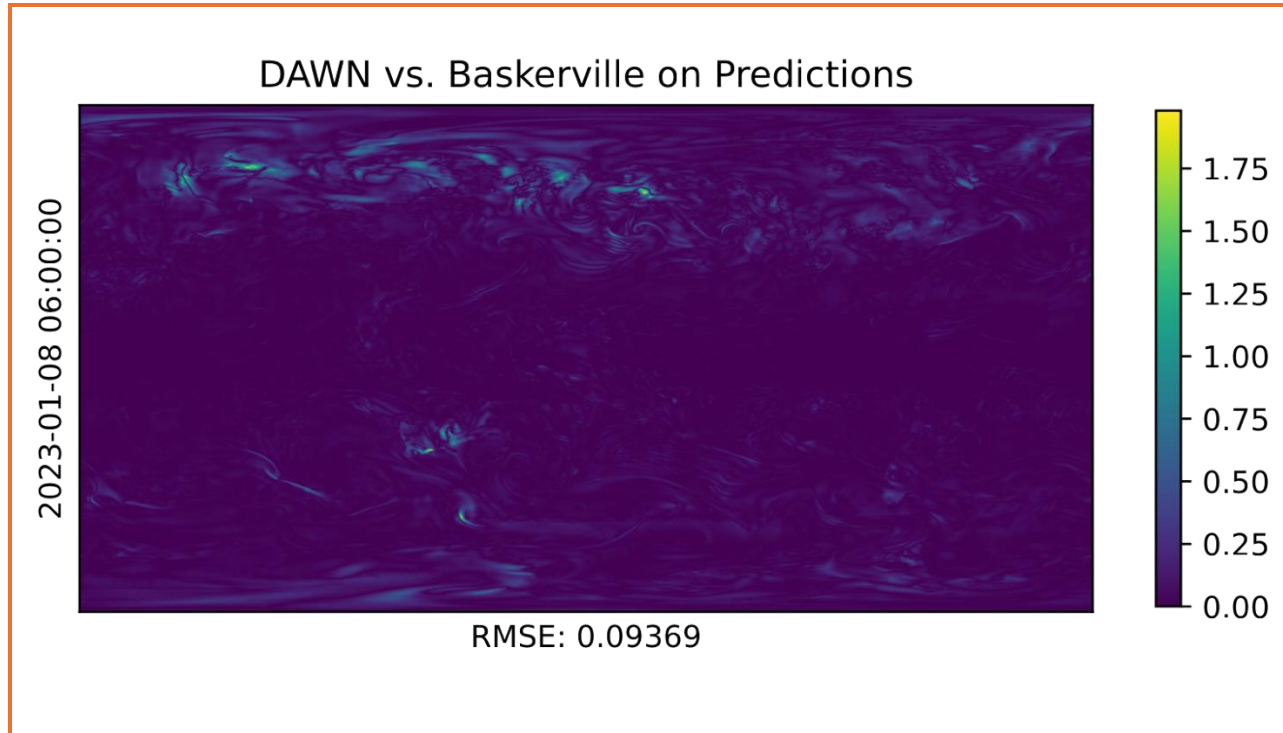
Inference results

Inference results

- 7 day rollout
- Prediction vs ground truth
- Two-meter temperature (1 10K range)



Inference results



Inference timing

- Average inference timing similar between Dawn and Baskerville

HPC	Hierarchy	Time for first step /s	Average time step /s	Time for 28 steps /s	Total run time /s
Dawn	<i>flat</i>	17.6	6.2	185.3	248.5
Dawn	<i>composite</i>	18.4	6.7	199.7	262.8
Baskerville	N/A	15.9	6.1	180.5	255.5

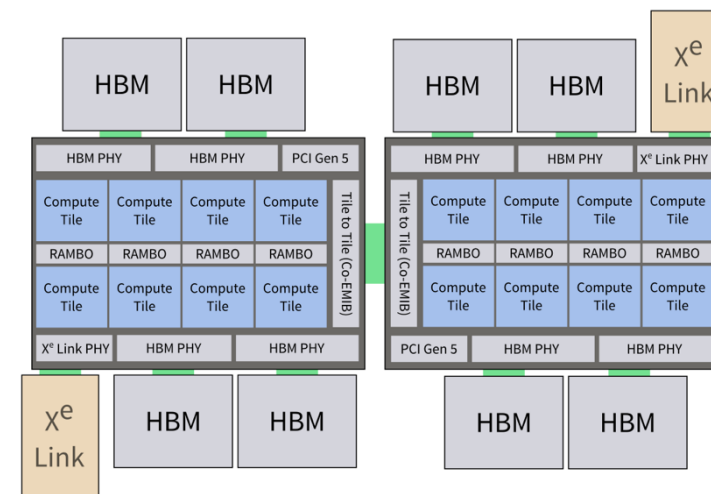
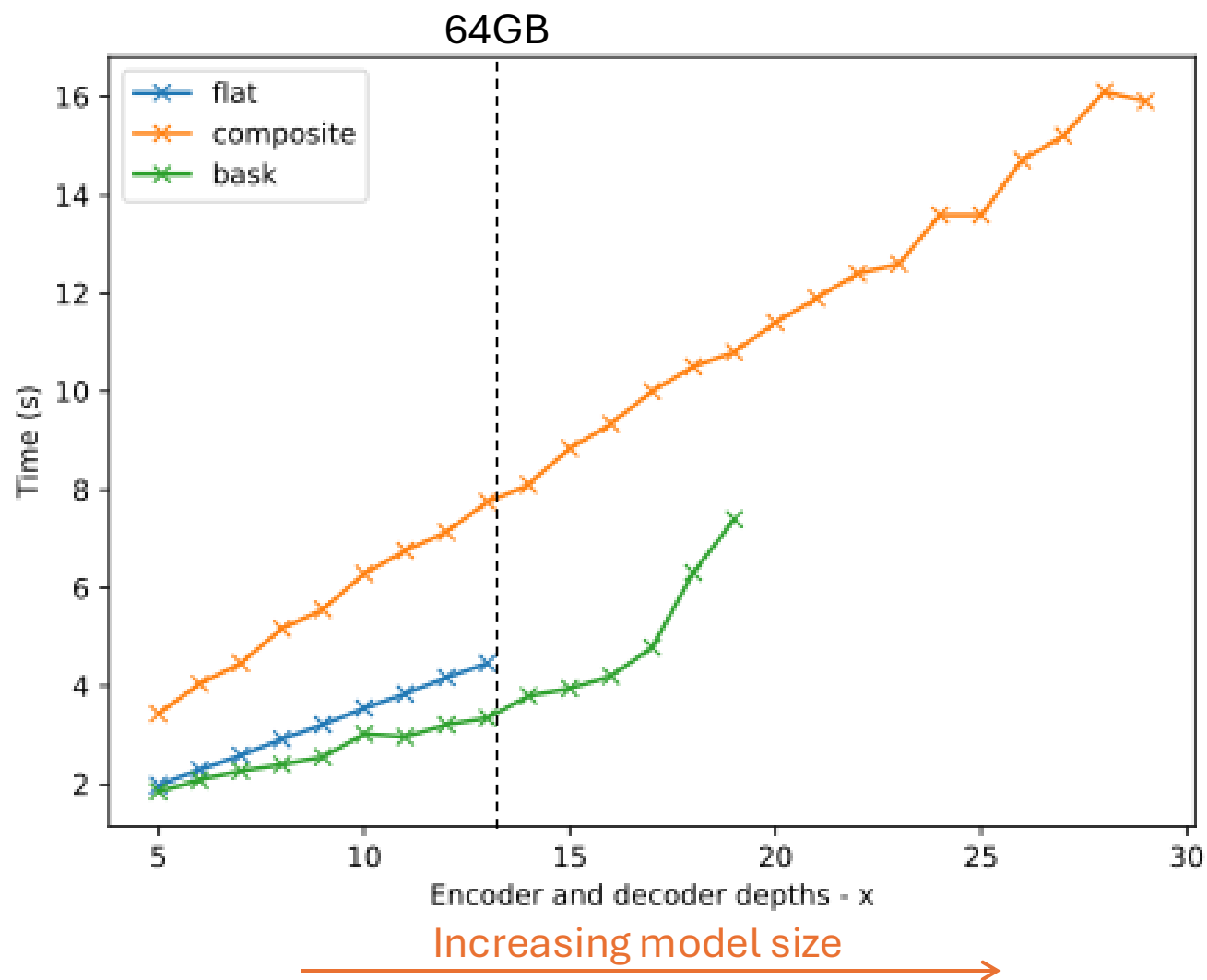
Training results

Training timing

Nodes	Total no. of GPUs	Dawn			Baskerville		
		Compute time per iteration /s	Wall-clock time per iteration /s	Total run time /s	Compute time per iteration /s	Wall-clock time per iteration /s	Total run time /s
1	1	18.13	18.13	636.14	5.88	5.88	250.55
1	4	17.76	4.44	212.96	6.83	1.71	120.34
2	4	19.4	4.85	198.54	6.33	1.58	116.46
4	4	19.6	4.9	201.28	6.22	1.56	117.84
2	8	19.18	2.43	122.76	6.92	0.87	94.4

← ~3-4x

Why?



Device hierarchy (*flat* vs *composite*)

GPUs	Torch Devices	Hierarchy	Compute time per iteration /s	Wall-clock time per iteration /s	Total run time /s
1	1	<i>flat</i>	6.86	6.86	257.43
1	2	<i>flat</i>	7.84	3.92	176.54
4	4	<i>flat</i>	8.13	2.03	122.72
4	8	<i>flat</i>	7.73	0.97	90.99
1	1	<i>composite</i>	11.1	11.1	391.7
2	2	<i>composite</i>	11.45	5.72	259.46
4	4	<i>composite</i>	11.54	2.89	152.56
1	1	N/A (Baskerville)	3.2	3.2	145.53
2	2	N/A (Baskerville)	3.25	1.62	104.85
4	4	N/A (Baskerville)	3.29	0.82	81.72

Same
no. of
devices

1 full
GPU

Conclusions

-  Relatively easy to port PyTorch code to Intel GPUs
 - Intel's oneAPI toolkit
 - Intel extension for PyTorch
 - move things to `xpu`
-  Inference results and timing approximately consistent between Dawn and Baskerville
-  Train ~3x slower on Dawn when using *composite* mode vs Baskerville
-  Train ~2x faster in *flat* vs *composite* mode on Dawn, approaching performance on Baskerville
-  Dawn team are looking into this more

Acknowledgements

- David Llewellyn-Jones, Iain Stenson and Tomas Lazauskas
- Dawn team
- Aurora team

A photograph of the Aurora Borealis (Northern Lights) over a calm lake at night. The aurora displays vibrant green and blue vertical streaks against a dark, star-filled sky. The lights are reflected in the still water below. Silhouettes of mountains and hills are visible in the background.

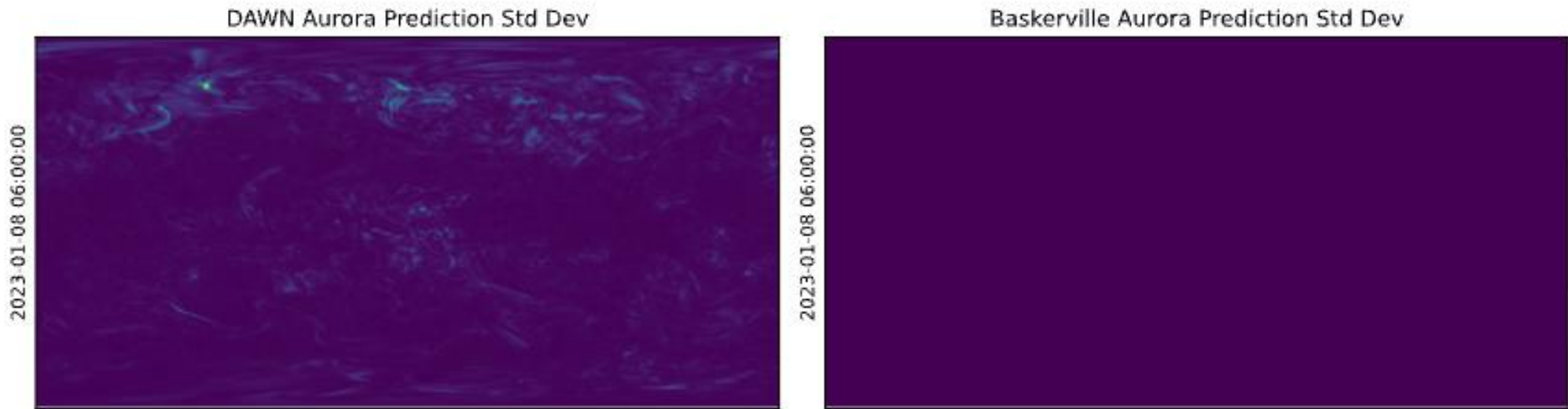
Questions?

Links and extra info

- <https://github.com/alan-turing-institute/aurora-hpc/tree/main>
- Report on this work coming soon (date TBC)
- <https://github.com/microsoft/aurora/tree/main>
- <https://docs.hpc.cam.ac.uk/hpc/user-guide/pvc.html>

Reproducibility

- Repeats on Dawn initially showed small variation in inference results
- Needed to set `torch.use_deterministic_algorithms(True)` to fix this



Training timing

Nodes	GPUs	Dawn			Baskerville		
		Compute time per iteration /s	Wall-clock time per iteration /s	Total run time /s	Compute time per iteration /s	Wall-clock time per iteration /s	Total run time /s
1	1	18.13	18.13	636.14	5.88	5.88	250.55
1	4	17.76	4.44	212.96	6.83	1.71	120.34
2	4	19.4	4.85	198.54	6.33	1.58	116.46
4	4	19.6	4.9	201.28	6.22	1.56	117.84
2	8	19.18	2.43	122.76	6.92	0.87	94.4